

Custom Performance Monitor and Frame Rate Manager for Unity

A lightweight, ultra-fast, and high-precision performance monitoring and frame rate management system designed specifically for Unity projects (2021.3+ / 2022.3+ / 2023+).

Built with zero dynamic memory allocation in mind, this package provides real-time telemetry for FPS, frametime, memory, and hardware specs without CPU overhead or Garbage Collector spikes.

☀ Key Features

- **Zero Garbage Collection (GC) Impact:** Uses pre-allocated buffers and optimized string formatting to prevent allocations in the main update loop.
 - **Precise Telemetry Tracking:** Real-time calculation of Average FPS, Frametime (ms), Min/Max FPS range, and Frame Stability ($\pm$ FPS).
 - **Native Hardware Insights:** Displays GPU, VRAM, CPU (core count), System RAM, and OS details using Unity's SystemInfo API.
 - **System Memory Tracking:** Measures allocated and reserved system memory using Unity's native ProfilerLong API.
 - **Flexible FPS & VSync Control:** Easily toggle between VSync, capped framerates (30 / 60 FPS), or uncapped performance via full UI Dropdown integration.
 - **Editor-Only Logging:** Employs the [Conditional("UNITY_EDITOR")] attribute to eliminate Debug.Log overhead in standalone builds.
 - **Native TextMeshPro Support:** Direct integration with TextMeshPro UI components.
 - **Cross-Platform & Modern Graphics APIs:** Full support for Vulkan, DirectX 11/12, Metal, and OpenGL on Linux, Windows, macOS, Android, and iOS.
-

📁 Package Structure

```
Assets/
├── CustomPerformanceMonitor/
│   ├── scripts/
│   │   ├── CustomPerformanceMonitor.cs
│   │   └── FrameRateManager.cs
│   ├── prefabs/
│   ├── Demo/
│   │   ├── scripts/
│   │   │   ├── PlayerMovement.cs
│   │   │   └── CameraFollow.cs
│   │   ├── character/
│   │   ├── mats/
│   │   └── Demo.unity
│   ├── CREDITS_AND_LICENSING.txt
│   └── README_Documentation.md
```

Quick Start Guide

1. Setting Up the Performance Monitor

1. In your Unity scene, create a UI Canvas containing two **TextMeshPro - Text (UI)** elements:
 - One text element for dynamic stats (FPS, Frametime, Memory).
 - One text element for static hardware specifications.
2. Add the CustomPerformanceMonitor script to an empty GameObject (e.g., [PerformanceMonitor]).
3. Drag and drop your TextMeshPro components into the respective fields in the Inspector:
 - **Dynamic Text:** Assign the dynamic stats text element.
 - **Static Specs Text:** Assign the hardware specs text element.
4. Set the **Update Interval** (default is 0.5s for optimal readability and zero impact on frame rendering).

2. Setting Up the Frame Rate Manager

1. Add the FrameRateManager script to a manager GameObject or the UI Canvas.
2. Link a UI **Dropdown (TMP)** to call FrameRateManager.SetFrameRateFromDropdown(int index) on its OnValueChanged event.
3. Dropdown index options correspond to:
 - 0: **VSync Enabled** (Adapts dynamically to 60Hz, 75Hz, 144Hz, or high-refresh rate monitors).
 - 1: **30 FPS Cap** (Ideal for mobile battery preservation or low-spec hardware).
 - 2: **60 FPS Cap** (Standard target for low input lag).
 - 3: **Uncapped / Unlimited** (Maximum hardware performance testing).

Script Reference & API

CustomPerformanceMonitor.cs

Monitors engine performance metrics and updates TextMeshPro UI controls at configurable intervals.

Public Fields

Field	Type	Description
dynamicText	TextMeshProUGUI	UI element displaying dynamic framerate and memory stats.
staticSpecsText	TextMeshProUGUI	UI element displaying system GPU, CPU, RAM, and OS information.
updateInterval	float	Refresh rate of the telemetry display in seconds (Default: 0.5f).

FrameRateManager.cs

Handles frame rate limits and VSync settings across desktop, mobile, and WebGL platforms.

Public Methods

```
// Changes framerate and VSync mode based on UI dropdown selection
public void SetFrameRateFromDropdown(int dropdownIndex);

// Sets exact target frame rate directly via code
public void SetFrameRate(int fps);
```

 Performance Considerations

- **Time.unscaledDeltaTime:** Performance measurements utilize unscaled delta time, ensuring calculations remain accurate even when game logic modifies Time.timeScale (e.g., pause menus or slow-motion effects).
- **Standalone Build Optimization:** All diagnostic log calls inside FrameRateManager and CustomPerformanceMonitor use [System.Diagnostics.Conditional("UNITY_EDITOR")]. In production builds, C# compiler optimizations strip these calls entirely, eliminating string allocations and console I/O.
- **Low Memory Footprint:** The included demo scene uses optimized Unlit/Texture shaders and low-poly geometry, keeping RAM consumption under **50 MB** in Standalone builds.

 Credits & Licensing Notice

- **Scripts & Implementation:** 100% Original C# Code written for Unity.
- **Demo Character Mesh & Textures:**
 - Character concept and orthographic reference designs created by the author.
 - Base 3D geometry generated using **Tripo AI (Pro Commercial License)**.
 - Rigging, topology refinement, UV adjustments, and Unity setup completed manually.
- **Licensing:** Granted for full commercial use in published games and applications.

*Created for Unity Asset Store compatibility.

Table of Contents

Custom Performance Monitor and Frame Rate Manager for Unity..... 1

 Key Features..... 1

 Package Structure..... 1

 Quick Start Guide..... 2

 Script Reference & API..... 2

 Performance Considerations..... 3

 Credits & Licensing Notice..... 3



Versión en Español: Monitor de Rendimiento Personalizado y Gestor de Cuadros para Unity

Un sistema ultrasensillo, ultra liviano y de alta precisión para el monitoreo de rendimiento y la gestión de tasas de refresco diseñado específicamente para proyectos en Unity (2021.3+ / 2022.3+ / 2023+).

Diseñado con cero asignación dinámica de memoria en mente, este paquete provee telemetría en tiempo real de FPS, tiempo por cuadro (frametime), memoria y hardware sin generar impacto de CPU ni picos en el Recolector de Basura (Garbage Collector).



Características Principales

- **Cero Impacto en Recolección de Basura (GC):** Utiliza búferes preasignados y formateo de cadenas optimizado para evitar asignaciones en el bucle principal.
- **Medición Precisa de Telemetría:** Cálculo en tiempo real de FPS Promedio, Tiempo por Cuadro (ms), Rango Mín/Máx de FPS y Estabilidad de Cuadros ($\pm FPS$).
- **Información Nativa del Hardware:** Muestra detalles de GPU, VRAM, CPU (conteo de núcleos), RAM del Sistema y Sistema Operativo usando la API SystemInfo de Unity.
- **Rastreo de Memoria del Sistema:** Mide la memoria asignada y reservada del sistema utilizando la API nativa ProfilerLong de Unity.
- **Control Flexible de FPS y VSync:** Alterna fácilmente entre VSync, cuadros limitados (30 / 60 FPS) o rendimiento ilimitado mediante integración completa con Dropdown de UI.
- **Registros Exclusivos para el Editor:** Emplea el atributo `[Conditional("UNITY_EDITOR")]` para eliminar el impacto de Debug.Log en ejecuciones independientes (Builds).
- **Nativo para TextMeshPro:** Integración directa con elementos de interfaz TextMeshPro.
- **Multiplataforma y APIs Gráficas Modernas:** Soporte completo para Vulkan, DirectX 11/12, Metal y OpenGL en Linux, Windows, macOS, Android e iOS.



Estructura del Paquete

```
Assets/
├── CustomPerformanceMonitor/
│   ├── scripts/
│   │   ├── CustomPerformanceMonitor.cs
│   │   └── FrameRateManager.cs
│   ├── prefabs/
│   ├── Demo/
│   │   ├── scripts/
│   │   │   ├── PlayerMovement.cs
│   │   │   └── CameraFollow.cs
│   │   ├── character/
│   │   ├── mats/
│   │   └── Demo.unity
│   ├── CREDITS_AND_LICENSING.txt
│   └── README_Documentation.md
```

Guía de Inicio Rápido

1. Configuración del Monitor de Rendimiento

1. En tu escena de Unity, crea un Canvas de UI con dos elementos **TextMeshPro - Text (UI)**:
 - Un texto para estadísticas dinámicas (FPS, tiempo de cuadro, memoria).
 - Un texto para especificaciones estáticas del hardware.
2. Agrega el script `CustomPerformanceMonitor` a un `GameObject` vacío (ej. `[PerformanceMonitor]`).
3. Arrastra y suelta los componentes `TextMeshPro` en los campos correspondientes del Inspector:
 - **Dynamic Text:** Asigna el elemento de texto para estadísticas dinámicas.
 - **Static Specs Text:** Asigna el elemento de texto para especificaciones de hardware.
4. Define el **Intervalo de Actualización / Update Interval** (el valor por defecto es 0.5s para una lectura óptima y cero impacto en el renderizado).

2. Configuración del Gestor de Tasa de Refresco (Frame Rate Manager)

1. Agrega el script `FrameRateManager` a un `GameObject` gestor o al Canvas de UI.
2. Vincula un **Dropdown (TMP)** de UI para llamar a `FrameRateManager.SetFrameRateFromDropdown(int index)` en su evento `OnValueChanged`.
3. Los índices del Dropdown corresponden a:
 - 0: **VSync Activado** (Se adapta dinámicamente a monitores de 60Hz, 75Hz, 144Hz o de alta frecuencia).
 - 1: **Límite de 30 FPS** (Ideal para ahorro de batería en móviles o hardware de gama baja).
 - 2: **Límite de 60 FPS** (Objetivo estándar con bajo retraso de entrada / input lag).
 - 3: **Sin Límite / Ilimitado** (Para pruebas de rendimiento máximo del hardware).

Referencia de Scripts y API

`CustomPerformanceMonitor.cs`

Monitorea las métricas de rendimiento del motor y actualiza los controles de UI de `TextMeshPro` a intervalos configurables.

Campos Públicos

Campo	Tipo	Descripción
<code>dynamicText</code>	<code>TextMeshProUGUI</code>	Elemento de UI que muestra los FPS dinámicos y métricas de memoria.
<code>staticSpecsText</code>	<code>TextMeshProUGUI</code>	Elemento de UI que muestra la información del sistema (GPU, CPU, RAM y SO).
<code>updateInterval</code>	<code>float</code>	Tasa de refresco de la pantalla de telemetría en segundos (Por defecto: 0.5f).

FrameRateManager.cs

Gestiona los límites de cuadros por segundo y configuraciones de VSync en plataformas de escritorio, móviles y WebGL.

Métodos Públicos

```
// Cambia los FPS y el modo VSync según la selección del dropdown de la UI public void  
SetFrameRateFromDropdown(int dropdownIndex);
```

```
// Define la tasa de refresco objetivo directamente por código public void SetFrameRate(int fps);
```

Consideraciones de Rendimiento

- **Time.unscaledDeltaTime:** Las mediciones de rendimiento utilizan el tiempo transcurrido sin escalar, garantizando que los cálculos sean precisos incluso si la lógica del juego modifica `Time.timeScale` (por ejemplo, menús de pausa o efectos de cámara lenta).
- **Optimización en Builds Independientes:** Todas las llamadas de registro de depuración en `FrameRateManager` y `CustomPerformanceMonitor` usan `[System.Diagnostics.Conditional("UNITY_EDITOR")]`. En ejecutable final, las optimizaciones del compilador de C# remueven completamente estas llamadas, eliminando la asignación de cadenas e I/O de consola.
- **Baja Huella de Memoria:** La escena demo incluida utiliza materiales optimizados Unlit/Texture y geometría low-poly, manteniendo el consumo de RAM por debajo de 50 MB en ejecutable Standalone.

Créditos y Licencia

- **Scripts e Implementación:** Código C# 100% original escrito para Unity.
- **Malla 3D y Texturas del Personaje Demo:**
 - El diseño conceptual y las referencias ortográficas fueron creados por el autor.
 - La geometría 3D base fue generada usando **Tripo AI (Licencia Comercial Pro)**.
 - El armado de rigging, retopología, ajustes de UVs y la integración en Unity se realizaron de forma manual.
- **Licencia:** Concedida para uso comercial completo en juegos y aplicaciones publicadas.

Documentación creada para compatibilidad con la Unity Asset Store.

Índice de temas

	Versión en Español:Monitor de Rendimiento Personalizado y Gestor de Cuadros para Unity.....	4
	Características Principales.....	4
	Estructura del Paquete.....	4
	Guía de Inicio Rápido.....	5
	Referencia de Scripts y API.....	5
	Consideraciones de Rendimiento.....	6
	Créditos y Licencia.....	6